Secure CAN Gateway — Full Design Report

- Plan: Final secure CAN design — Core1 (capture/count/forward) + Core2 (validate/log), detection-not-prevention goal
  - Design History & Decision Report
    - 1. What the project had before this session
    - 2. The professor’s actual, verbatim-quoted correction
    - 3. Every design tried against that bar, and exactly why each failed
    - 4. Why Design 8 succeeds where 2 through 7 all failed
  - Technical Audit & Critique: Latency Realities & SPI Mechanics
    - 1. Verified Realities in Code
    - 2. Corrections & Flaws Addressed
  - Context & Architecture
    - Accepted Detection Window
  - What will be built
    - 1. Divert Core2 into Secure World
    - 2. Wire Up Core1 → Linux Fast-Path (First-Time Implementation)
    - 3. SPI Transaction Collapse (9 → 3–4 Transactions with Error Fallback)
    - 4. Core1 → Core2 Cross-Core Secure Queue
    - 5. Core2 Crypto Validation & Tamper-Evident Log
  - Verification Plan
  - What’s Still Genuinely Open — Real Implementation Work, Not Design Flaws
  - Backup Checkpoint (before touching real hardware)
  - Open Items & Communication Points for the Professor

Plan: Final secure CAN design — Core1 (capture/count/forward) + Core2 (validate/log), detection-not-prevention goal

Design History & Decision Report

1. What the project had before this session

Secure World (Core1) already owned the CAN controller directly: the interrupt from the MCP2518FD (GPIO25) routed straight to Secure World, which validated every frame (HMAC-SHA256 + counter continuity) *before* Linux ever saw it, then handed the result to Core0/Linux.

Why the professor pushed back on this: latency. Every single frame pays the full crypto-validation cost before Linux gets it. His stated objection: “it increases latency.”

2. The professor’s actual, verbatim-quoted correction

> “…packet of controller goes into normal world normally, secure world monitors every packet which will come in the system of related of can controller — if packet is corrupted, make a log of it. I have not prevented the attacker from touching the packet — if they do touch it, I have a secure log in flash.”
>
> “…secure world works like previous — no Linux touch that secure world.”

This set the real bar for the whole rest of the session: **detection, not prevention.** Linux can get the frame fast (fixing his latency complaint); Secure World’s job is to notice and log tampering after the fact, not block it. It also reconfirmed that Secure World must stay Linux-untouchable, same as the existing proven design.

3. Every design tried against that bar, and exactly why each failed

#	Design	Why it failed
1	**Old design**: Secure World validates every frame before Linux sees it	Rejected by the professor himself — too slow (crypto on the critical path for every frame)
2	**Linux-owns-hardware + shared-memory monitoring**: frame goes to Linux first; Secure World watches a copy in shared memory	Fast, but reopens the **omission gap**: a compromised Linux can simply not write a frame into shared memory. Secure World never even learns the frame existed — catches corruption, not omission
3	**CS-line/GPIO watching**: Secure World watches the SPI chip-select/interrupt line itself to notice “a transaction happened”	Only proves *something* happened on the bus, not *what* — Linux still owns and formats the data, so this doesn’t close the gap
4	**Independent controller-register peek** (incl. the timer-driven “ask the controller for its counter” variant): Secure World queries the CAN controller’s own counter register on a timer, independent of Linux	Two separate blockers: (a) **SPI bus collision** — Linux also owns SPI0 in this design, no hardware arbiter exists, so Secure World’s query can corrupt a live Linux transaction or vice versa; (b) the register’s actual semantics were never verified (true cumulative counter vs. a FIFO fill-level that drains/resets)
5	**Mandatory kernel-driver-level push**: make the kernel driver structurally always push every frame to Secure World	Only defeats a userspace-only compromise — a fully compromised kernel can just skip the push call. Partial fix, not a closed gap
6	**Dynamic runtime SPI0/GPIO25 ownership handover**: hand the hardware back and forth between Linux and Secure World based on a liveness timer	Rejected outright — requires live TF-A/interrupt-routing register changes with no safe window, the same class of change that caused this project’s real board-hang incident. Also adds a new failure mode (the handover logic itself)

#	Design	Why it failed
7	**External microcontroller / second physical CAN channel (`can1`) as an independent witness**	Would work in principle, but `can1` has an unresolved hardware fault on this board (`osc=0x0` , confirmed not a device-tree, driver, or kernel-version bug via direct source comparison — likely physical/electrical, needs hands-on continuity testing). Also simply unnecessary once Design 8 below is found
8	**Core1 (capture/count/forward) + Core2 (validate/log)** — see below	**This is the one that works.** No patch, no workaround — it restructures the problem so the omission gap never opens in the first place

4. Why Design 8 succeeds where 2 through 7 all failed

Designs 2–7 all tried to let Secure World *independently verify* a setup where **Linux touches the hardware first**. That ordering is exactly what creates the omission gap — verification-after-the-fact can always be defeated by not reporting at all.

Design 8 removes the ordering problem entirely instead of patching around it: **hardware ownership never moves to Linux.** It stays exactly where it already is today (Core1, Secure World, SPI0+GPIO25 — unchanged from the current proven architecture). Core1 captures every frame, fires it to Linux immediately with zero crypto (fixes the professor's latency objection), and *separately* hands a copy to a second dedicated Secure-World core, Core2, which does the HMAC+counter validation and tamper-evident logging off to the side, never touching Linux.

Because Core1 is still the *first and only* thing that ever touches the wire — exactly like today — there is no point at which a compromised Linux could ever hide a frame from Secure World's knowledge. This isn't best-effort detection; it's structurally closed, because Secure World's knowledge of what arrived never depends on Linux's cooperation at all.

Technical Audit & Critique: Latency Realities & SPI Mechanics

A rigorous source audit of `rpi3_can_fiq.c` , `rpi3_can_server.c` , and the MCP2518FD hardware architecture verified several key facts and corrected earlier oversimplifications:

1. Verified Realities in Code

- **`SPI_CLK_DIVIDER = 32` is real (`rpi3_can_fiq.c:376`)**: At BCM2837 core clock 250MHz, the SPI bus runs at ~7.8MHz (~1.02 µs/byte).
- **The "9 SPI transactions per frame" is verified**:
  - `read_pending_rx_frame()` does 7 transactions: `REG_RXIF` read, `REG_INT` read, `REG_BDIAG1` read, `FIFOUA` read, message-RAM object read, `FIFOCON` read, `FIFOCON` write (UINC).
  - `interrupt_main_handler()` does 2 transactions: `REG_INT` read, `REG_INT` write-to-clear.
  - Total = 9 separate CS assertion cycles per frame.
- **Single-slot `can0_frame_state` is real**: Causes head-of-line blocking under burst traffic, and `can0_frame_dropped_count` exists specifically to count frames dropped when the slot is busy.

- `wait_for_spi_idle()` **is safety-critical**: Added after Incident 9 (KNOWLEDGE.md) to prevent bus collisions if Linux's SPI0 driver is mid-transaction. It **must remain** at the entry of the capture sequence.
- **Hardware FIFO Depth Headroom**: `CAN0_CAPTURED_RXFIFOCON` sets `31U << MCP251XFD_FIFOCON_FSIZE_SHIFT` (32 message slots total in hardware FIFO 1). Delaying `FIFOCON` UINC until after the software push to Linux introduces negligible overflow risk under burst traffic, as the hardware FIFO easily buffers up to 32 frames while the push takes only microseconds.

2. Corrections & Flaws Addressed

- **Correction 1:** `push_result()` **was never wired up**: `rpi3_can_server_push_result()` is defined in `rpi3_can_server.c`, but grep confirms **zero call sites** across the entire codebase. This matches the commit history ("Step 16/17 known frame-content bug, not yet fixed"). Right now there is no Core1→Linux push in operation. The task is **building and wiring up the Core1→Linux forwarding pipeline for the first time**, not merely reordering an existing working path.
- **Correction 2: "Single 62-byte burst read" is physically impossible on MCP2518FD**:
  - **Address gap**: `INT` (0x001c), `RXIF` (0x0020), `BDIAG1` (0x003c), `FIFOUA` (0x0058+0xc*fifo) are in SFR space. Message RAM begins at 0x0400 (a gap of ~996 bytes). An SPI burst auto-increments contiguous memory; it cannot jump SFR space to RAM in one command.
  - **Dynamic pointer indirection**: `addr = CAN0_MSGRAM_BASE + (offset read from FIFOUA)`. The message address is dynamic and assigned by the controller's FIFO engine. You cannot address a burst read to an address not yet known.
  - **Physical minimum**: Reading a frame strictly requires **two sequential, dependent SPI transactions**:
    1. Read `FIFOUA` (and status).
    2. Read the Message RAM object at `CAN0_MSGRAM_BASE + offset`.
  - **Honest achievable target**: Collapsing 9 transactions down to **3–4 transactions** on the clean path.
- **Correction 3: BDIAG1 / Error Diagnostics Fallback & Missing #defines**:
  - `BDIAG1` latches error bits (DLCMM, NACKERR, form/stuff/CRC errors) and must be read before being cleared.
  - Cross-check against `linux/drivers/net/can/spi/mcp251xfd/mcp251xfd.h`:

```
#define MCP251XFD_REG_INT_IVMIF  BIT(15)
#define MCP251XFD_REG_INT_CERRIF BIT(13)
```

  - In `rpi3_can_fiq.c`, only `RXIF` (`BIT32(1)`) and `RXIE` (`BIT32(17)`) currently exist. The plan explicitly specifies adding:

```
#define MCP251XFD_REG_INT_IVMIF  BIT32(15) /* Invalid Message Interrupt Flag */
#define MCP251XFD_REG_INT_CERRIF BIT32(13) /* CAN Bus Error Interrupt Flag */
```

  - **Branching Policy**: Transaction 1 reads `REG_INT` and `FIFOUA`. If `INT & (MCP251XFD_REG_INT_IVMIF | MCP251XFD_REG_INT_CERRIF) == 0`, proceed directly on the clean fast path (skipping `BDIAG1`). If an error bit is set, take the **diagnostic fallback path**: immediately

read `BDIAG1` into `can0_last_bdiag1` before clearing or advancing the chip, preserving full diagnostic visibility without penalizing normal frames.

- **Correction 4: Latency Figures are Modeled Targets, Not Settled Results**: Per the project's roadmap rule (*"don't claim latency numbers without measuring them"*), all figures such as "< 25–50 µs total" or "~20 µs Core 1 time" are **modeled engineering targets**. Concrete numbers will be established via hardware PMCCNTR cycle-counter benchmarking in Step 4 of the Verification Plan.
- **Correction 5: Software Polling Overhead vs. Clock Speed**: `spi_wait_bit()` and `wait_for_spi_idle()` are busy-wait CPU loops (`for (i = 0; i < SPI_POLL_TIMEOUT_LOOPS; i++) { if (bit set) return; }`). For a 4-byte register read, wire time at 7.8MHz is only ~4 µs. The bulk of the measured delay across 9 transactions is fixed software polling loops, MMIO register access overhead, and redundant GPIO pin-muxing in `can0_cs_set()`. Cutting transaction count from 9 to 3–4 attacks the true dominant bottleneck.

Context & Architecture

The architecture retains **Secure World (Core1) hardware ownership** (preventing the omission gap), and splits duties across dedicated cores:

```
 Core1 (dedicated, S-EL1): owns SPI0/GPIO25 directly (UNCHANGED ownership)
    → asserts wait_for_spi_idle() (preserves Incident 9 bus safety)
    → captures frame via optimized 3–4 transaction SPI sequence
    → checks INT flags: if IVMIF/CERRIF asserted, reads BDIAG1 (diagnostic fallback)
    → increments ground-truth counter
    → FAST-PATH: hands copy to Linux via shared coherent SPSC ring buffer & doorbell
    → hands copy to Core2 via cross-core queue
    → completes post-forward housekeeping (FIFOCON UINC, INT clear)

Core2 (dedicated, S-EL1, newly diverted via TF-A PSCI intercept):
    → dequeues frame from Core1
    → recomputes HMAC-SHA256(key, id||counter||payload), compares to received tag
    → checks counter continuity & timestamp freshness
    → on failure, appends to tamper-evident hash-chained log
```

Accepted Detection Window

Linux consumes and acts on the frame **~100 µs target** (pending live PMCCNTR measurement) before Core2 finishes HMAC-SHA256 verification. This is the **explicitly accepted and documented tradeoff** required by the professor's mandate ("detection, not prevention; packet goes into normal world normally").

What will be built

1. Divert Core2 into Secure World

- Replicate `opteed_enter_secure_server_if_core1()` in TF-A (`opteed_pm.c`) for Core2's MPIDR/position.
- Intercept Core2 at `bl31_warm_entrypoint` tail before Non-Secure `eret`.
- Core2 enters a permanent `__noreturn rpi3_can_server_core2_loop()` in OP-TEE core, unmasking native exceptions (`thread_set_exceptions`).

2. Wire Up Core1 → Linux Fast-Path (First-Time Implementation)

- Wire up the actual invocation of `rpi3_can_server_push_result()` (previously uncalled).
- Upgrade the single-slot buffer into a **lock-free Single-Producer Single-Consumer (SPSC) circular ring buffer** (64 slots) in shared coherent DRAM (`reserved-memory` DT node).
- Core1 writes `slots[head]`, executes memory barrier (`dsb st`), advances `head`, and triggers Mailbox 1 doorbell to wake Core0/Linux.
- Ordering rule: Core1 pushes to Linux **immediately after reading the message RAM object**, *before* executing the post-forward SPI transactions (FIFOCON UINC and INT clear). Hardware FIFO 1's 32-slot depth guarantees safe buffering during this short handoff.

3. SPI Transaction Collapse (9 → 3–4 Transactions with Error Fallback)

- **Add Required Register Bit Definitions** to `rpi3_can_fiq.c`:

```
#define MCP251XFD_REG_INT_IVMIF  BIT32(15) /* Invalid Message Interrupt Flag */
#define MCP251XFD_REG_INT_CERRIF BIT32(13) /* CAN Bus Error Interrupt Flag */
```

- **Entry Check**: Maintain `wait_for_spi_idle()` prior to SPI access (Incident 9 safety).
- **Transaction 1 (Status & Pointer)**: Read `FIFOUA(1)` and `REG_INT`.
  - *Clean Fast Path*: If `(int_reg & (MCP251XFD_REG_INT_IVMIF | MCP251XFD_REG_INT_CERRIF)) == 0`, proceed directly to Transaction 2.
  - *Diagnostic Fallback*: If `IVMIF` or `CERRIF` is set, execute an immediate read of `REG_BDIAG1` into `can0_last_bdiag1` before any further access.
- **Transaction 2 (Message Object)**: Read 60-byte CAN-FD object at `CAN0_MSGRAM_BASE + ua`.
- *[Fast-Path Push to Linux Happens Here]*
- **Transaction 3 (FIFO Advance)**: Write `FIFOCON(1)` with `UINC` bit.
- **Transaction 4 (Interrupt Acknowledge)**: Write `REG_INT` to deassert `/INT`.
- **Clock & Pin-mux optimization**:
  - Increase SPI clock divider (`SPI_CLK_DIVIDER 16` or `12`, moving from 7.8MHz to 15.6–20MHz).
  - Remove redundant `GPIO_GPFSEL0` read-modify-writes inside `can0_cs_set()` (initialize pin function once during boot).

4. Core1 → Core2 Cross-Core Secure Queue

- Dedicated SPSC queue in Secure World RAM between Core1 and Core2.
- Core1 enqueues frame copy; Core2 drains and processes asynchronously.

5. Core2 Crypto Validation & Tamper-Evident Log

- Core2 executes `can0_compute_hmac()` (HMAC-SHA256) and checks counter continuity (`secure_last_counter`).
- On failure: appends entry to hash-chained log (`H_i = HMAC(key, H_{i-1} || entry_i)`).
- Tip maintained in Secure World RAM (explicitly documented as RAM-only / lost on reboot).

Verification Plan

1. **Isolation of Core2 Diversion**: Boot with trivial heartbeat on Core2. Confirm via UART that Core2 enters Secure World and never reaches Linux.
2. **Wire Up & Verify Core1→Linux Push**: Confirm for the first time that Core0 receives frames from Core1 via the SPSC ring buffer and Mailbox 1 doorbell.
3. **SPI Transaction Reduction & Stability**: Verify on hardware that the 3–4 transaction sequence correctly drains FIFO 1 without bus errors or missed interrupts while preserving `wait_for_spi_idle()`.
4. **Diagnostic Fallback Verification**: Force an invalid CAN frame (e.g. invalid DLC or CRC error); verify that `INT` flags (`IVMIF` / `CERRIF`) trigger the diagnostic branch and `BDIAG1` is correctly captured into `can0_last_bdiag1`.
5. **Empirical Latency Measurement (ARM PMCCNTR Cycle Counter)**: Measure concrete elapsed CPU cycles from GPIO25 FIQ entry to Core0 Linux reception. Compare against baseline to validate whether the theoretical target (< 25–50 µs) is achieved.
6. **Detection Window Measurement**: Measure empirical time delta between Linux frame consumption and Core2 validation verdict (~100 µs target).
7. **Tamper Log Verification**: Inject corrupted frame; confirm Linux receives it without delay while Core2 appends a tamper entry to the hash chain. Corrupt a historical entry and confirm hash-chain breakage.
8. **Burst / Overflow Testing**: Stream CAN frames at 1000+ frames/sec; verify that hardware FIFO 1 (32 slots) and software SPSC ring buffer (64 slots) maintain zero drops.

What's Still Genuinely Open — Real Implementation Work, Not Design Flaws

1. **Core2's TF-A diversion code doesn't exist yet** — new code, parallel to Core1's proven mechanism, not yet written.
2. **Core2's permanent loop doesn't exist yet** — new function, not yet written.
3. **The Core1→Core2 queue doesn't exist yet** — only the Core1→Core0 single-struct mechanism currently exists; a second instance targeting Core2 is new.
4. **The 64-slot ring buffer is unproven on this hardware** — the only proven cross-core mechanism so far is a single struct + doorbell, not a multi-slot ring; needs its own reserved-memory DT node and a check of Secure/Normal-World memory-attribute (cache/shareability) behavior, since nothing in this project has moved multi-slot shared state across that boundary before.
5. **The `IVMIF` / `CERRIF` `#define`s are verified correct but not yet typed into the file.**

6. **Any TF-A/PSCI change here must never be flashed without UART connected** — this exact class of change (warm-boot/interrupt-routing) caused this project's prior board-hang incident; Core2's diversion touches the same code path.
7. **The push-before-UINC ordering change is only checked on paper (FIFO-depth math)** — still needs a real burst test on hardware before being called safe, not just plausible.
8. **Real latency measurement (PMCCNTR) is still outstanding** — the whole latency argument to the professor depends on this being measured, not assumed.
9. **Queue overflow policy for Core1→Core2** is proposed ("drop-oldest-and-log-the-drop") but not concretely sized or implemented.
10. **Log reboot-persistence** is deferred — RAM-only for now; a persistent/rollback-protected anchor (e.g. an external secure element like an ATECC608) is explicit future work, not required for this milestone.
11. **`can1`'s hardware fault** remains open but explicitly non-blocking for this design's security goal; the only remaining diagnostic step is a hands-on physical continuity/voltage check, which requires the user's own action.
12. **Core1 self-integrity check (optional addition)** — a timer+register-query mechanism, unsafe as a Linux-honesty check (bus collision risk in the old Linux-owns-hardware design) but safe here as a Core1 self-check against its own missed-interrupt bugs, since Core1 remains sole SPI0 master. Not yet added as a numbered build step — open question whether to include it.

Backup Checkpoint (before touching real hardware)

Before implementing any of the "What will be built" section above: copy the current SD card image to `SD_Card_Backups/pre-two-core-secure-split/rpi3-sdcard-<descriptive-name>.img` (confirm build-output-copy vs. live-device-read convention first, matching an existing checkpoint's method), add a short `KNOWLEDGE.md` entry in the same style as existing checkpoints (`6e582f9`, `0259424`), commit just that documentation addition.

Open Items & Communication Points for the Professor

- **Empirically Measured Latency**: Present the concrete PMCCNTR cycle-counted numbers rather than assumed estimates, demonstrating the speedup of the 3–4 transaction cut-through pipeline while proving hardware ownership remains isolated in Secure World.
- **Explicit ~100 µs Detection Window**: State clearly that Linux acts on frames ~100 µs before Core2 finishes validation, reflecting the "detection, not prevention" mandate.
- **Log Reboot-Persistence**: Log tip resides in Secure RAM; persistent secure storage (e.g. RPMB / external secure EEPROM) remains separate future work.
- **can1 Hardware Fault**: Open physical/electrical issue (`osc=0x0`), non-blocking for this single-channel gateway.